Objectif et modèle conceptuel

1. Objectif et périmètre

Le moteur transforme un document source de Modules et de Nodes en un **AST logique indépendant des langages cibles**, puis en informations sémantiques et diagnostics. Il accepte les étapes intermédiaires de construction : une erreur ne détruit ni la source ni la possibilité d'inspecter l'arbre.

Le périmètre livré couvre la construction de l'AST, les portées lexicales, les références locales et inter-modules, un typage statique strict et la sérialisation JSON versionnée. Le moteur est du JavaScript pur, sans React, DOM, Neutralinojs ou dépendance npm. L'interface graphique existante reste inchangée et n'est pas encore branchée au moteur.

Exclus : simulation, suggestions contextuelles, génération de code, analyse de flot avancée, modèles objets, persistance sur disque et éditeur spécifique à chaque construction. Le document décrit des fondations, pas un compilateur complet.

2. Concepts et relations distinctes

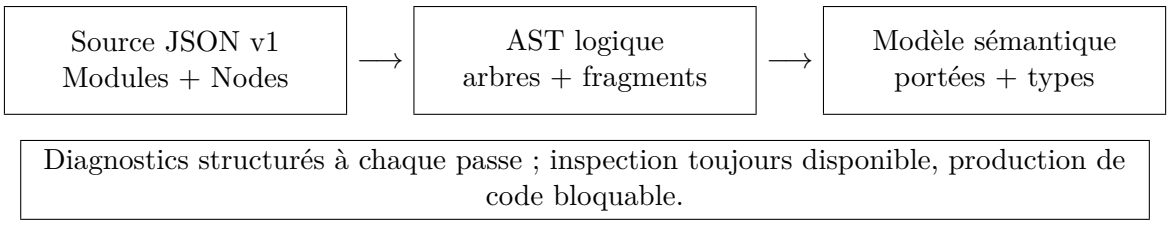
Project	Document source portant un identifiant, une version de format, une liste de modules et une liste globale de nœuds.
Module	Unité logique avec identifiant stable, racines syntaxiques, imports et exports explicites. Aucun lien obligatoire avec un fichier physique.
Node	Enregistrement discriminé par `kind`, propriétaire `moduleId`, propriétés, enfants syntaxiques et références symboliques.
AST	Arborescence dérivée : les identifiants d'enfants sont remplacés par des constructions hiérarchiques. Les `sourceId` restent stables.
Scope	Portée lexicale avec parent ; elle ne correspond pas nécessairement au module propriétaire.
Symbol	Déclaration accessible depuis une portée, ou alias d'import, associé à son nœud et à un type statique résolu ; un échec reste diagnostiqué sans type de secours.

Trois relations indépendantes : appartenance par `moduleId`, imbrication par `children`, référence par `references.target`. Une référence vers une déclaration ne la transforme pas en enfant de l'appelant. Une connexion graphique n'a donc aucun sens logique implicite.

Les catégories source `module` et `main` sont reconnues ; Main ne déclenche aucune exécution. Un projet valide possède zéro ou un seul Main ; chaque Main supplémentaire est diagnostiqué sans suppression. Les sous-modules ne sont pas introduits à ce stade.

Architecture et invariants

3. Flux et responsabilités



`model.js`, `nodeSchema.js`	Contrats JSDoc, fabriques sans génération d'identifiant, catégories, slots et opérateurs centralisés.
`ast.js`	Indexation, reconstruction des enfants, contrôle de structure, cycles, parents multiples et propriétaires ; parcours et fragments détachés.
`semantics.js`	Collecte des portées et déclarations, contrôle des imports/exports et résolution des références.
`typeAnalysis.js`	Annotations obligatoires, signatures et expressions statiquement déterminées, sans conversion implicite ni type de secours.
`primitives.js`, `numericConstants.js`	Catalogue de dix primitifs, bornes, arrondis et contrôles des expressions constantes.
`diagnostics.js`	Codes stables, gravité, message et localisation.
`serialization.js`	Lecture/écriture JSON sans migration implicite.

Tous ces fichiers sont dans `src/utils/logic/` ; l'API publique est `index.js`. La fonction `analyzeProgram(source)` renvoie `source`, `ast`, `semantic`, `diagnostics` et `canGenerate`. Les fonctions ne modifient pas la source.

4. Structures et invariants

- Identifiants de nœuds uniques dans le projet ; identifiants d'import uniques dans leur module. Aucun identifiant n'est recréé pendant les passes.
- Un enfant syntaxique a au plus un parent et appartient au même module. L'ordre des listes de slots est significatif, pas celui du tableau `nodes`.
- Déclarations : variables, constantes, fonctions, paramètres et alias de types. Instructions : blocs, retours, affectations, conditions et boucles. Expressions : littéraux, références, appels et opérations unaires/binaires.
- Un slot requis absent produit `Hole`. Un lien impossible produit `Invalid`. Les fragments détachés restent visibles ; la source conserve également les enregistrements malformés ou dupliqués.
- Annotations obligatoires des déclarations et littéraux : `primitive` ou alias transparent `named`. Signature `function` dérivée pour les fonctions. Dix primitifs : `short int`, `short uint`, `int`, `uint`, `float`, `double`, `string`, `char`, `boolean`, `void` (retour uniquement). Aucune promotion implicite.

Exemple et résolution des références

5. Deux modules, une référence intermodule

Le scénario exécutable `twoModules()` dans `tests/logic/fixtures.js` crée la constante publique `answer = 42` dans `math`, puis la variable `result` dans `app`.

```
Project { formatVersion: 1, id: "project" }
  Module "math"
    rootNodeIds: ["answer"]
    exports: [{ nodeId: "answer", name: "answer" }]
  Module "app"
    rootNodeIds: ["result"]
    imports: [{ id: "answerImport", moduleId: "math",
                nodeId: "answer", localName: "answer" }]

Node "answer" : ConstantDeclaration, moduleId = "math"
  properties: { name: "answer", type: primitive("int") }
  children.initializer: "fortyTwo"
Node "fortyTwo" : Literal, value = 42, type = primitive("int")

Node "result" : VariableDeclaration, moduleId = "app"
  properties: { name: "result", type: primitive("int") }
  children.initializer: "answerReference"
Node "answerReference" : Reference, moduleId = "app"
  references.target: { kind: "import", importId: "answerImport" }
```

`primitive("int")` est ici une notation abrégée pour `{kind:"primitive", name:"int"}`.

L'AST simplifié est le suivant ; chaque construction conserve son identifiant source :

```
Project
  Module math
    ConstantDeclaration [answer]
      initializer -> Literal [fortyTwo] : int
  Module app
    VariableDeclaration [result]
      initializer -> Reference [answerReference] : int
```

La déclaration `answer` n'est pas recopiée sous `result`. Le modèle sémantique associe la référence au symbole importé et au nœud cible `answer`. Sans export ou import explicite, la référence est rejetée.

Règles de portée : une portée par module, fonction, bloc imbriqué et `for`. Paramètres et bloc principal d'une fonction partagent une portée. Les noms sont collectés avant résolution ; un doublon local est une erreur, un masquage dans une portée interne est autorisé. Une référence directe par identifiant ne contourne pas les frontières lexicales ou les imports.

Vérification et évolutivité

6. Validation et résultats constatés

Un diagnostic contient `code`, `severity`, `message` et `location = {projectId, moduleId, nodeId, field}`. La localisation désigne le nœud fautif ou le propriétaire d'un slot manquant. Les éléments inapplicables sont nuls. Les codes ne dépendent pas de la langue du message.

Contrôles structurels : identifiants invalides/dupliqués, enfant absent, mauvais propriétaire, catégories illégales, cycles, parents multiples, fragments détachés et slots incomplets. Contrôles sémantiques : référence inaccessible, import/export invalide, conflit de déclaration, retour hors fonction, contrôle de boucle hors boucle, affectation d'une constante.

Typage strict : annotations, initialisation, opérateurs, conditions booléennes, signatures/appels, affectations et retours. Les expressions non typables restent inspectables sans type publié. Le contrôle de retour accepte les blocs et les deux branches d'un `if`, mais ne prouve pas les retours via les boucles. Entiers signés/non signés de 16/32 bits, flottants IEEE 754 de 32/64 bits ; `char` est un scalaire Unicode. Les conversions `CastExpression` sont numériques explicites, avec troncature ou arrondi et contrôle des bornes.

Vérifications exécutées : `pnpm test`, runner natif Node : **60 tests réussis, 0 échec**. Couverture de l'unicité de Main, des modules vides, déclarations multiples, fonctions/paramètres/retours, conditions et boucles imbriquées, références locales, import intermodule, références absentes, conflits, programmes incomplets, erreurs de type et reconstruction JSON. Des tests supplémentaires contrôlent les identifiants, localisations, sources gelées, fragments détachés, champs malformés, profondeurs, bornes numériques, sous-flux, arrondis difficiles et conversions interdites.

`pnpm build` : frontend compilé avec succès. Ce contrôle protège le frontend existant ; le moteur séparé est vérifié par les tests Node, pas par la compilation du bundle. Seules les catégories Module/Main sont partagées avec l'UI. Aucune nouvelle fonction native n'a été ajoutée ; pas de nouvelle vérification Neutralinojs.

7. Évolution sans couplage prématuré

La simulation pourra consommer l'AST et les symboles résolus, puis publier des événements indexés par `sourceId`. Les suggestions exploiteront les portées et les types au point d'édition. Chaque générateur abaissera ensuite les mêmes constructions vers sa cible ; aucun AST Babel n'est utilisé.

Un futur générateur devra refuser les diagnostics bloquants. `canGenerate` signifie seulement « aucune erreur détectée par les passes actuelles », et non une garantie d'exécutabilité. Aucun de ces trois moteurs n'est implémenté et aucun système de plugins n'a été inventé.

Le format v1 préserve les champs supplémentaires. Une version inconnue est conservée et signalée, sans migration. La sérialisation refuse les données non JSON au lieu de les supprimer silencieusement. Les profondeurs d'analyse et de sérialisation sont bornées pour éviter les dépassements de pile.

Limites assumées : pas de sous-modules, objets, génériques, réexports ou résolution de fichiers. Les dépassements constants sont diagnostiqués ; les valeurs reçues à l'exécution ne sont pas bornées globalement par cette analyse. Les futurs backends devront appliquer les mêmes opérations contrôlées, sans rebouclage. L'API complète et les conventions de slots sont décrites dans `docs/LOGIC_API.md`.